Security Verification of AI Agent Decommissioning and Offboarding

A Reproducible Study of Residual Effects, Checkpoint Races, and Recovery

Ezekiel Ologunde

ABSTRACT

Retirement acknowledgements do not establish that an agent has lost authority or that its residual state has been cleared. We develop a reproducible conformance framework that separately observes effects, cleanup, evidence quality, and recovery. A 720-run local microbenchmark adapts twelve public AgentTrap records into 24 paired synthetic scenarios under six controls; a separate 720-run replication agrees on all semantic outcomes. A root-scoped sink fence rejects every retired-root attempt while preserving an independent root, but failed cleanup leaves residuals in twelve distinct scenarios. Deliberately stale evidence falsely certifies four of them. An 80-run integration with LangGraph and APScheduler extends the test to actual checkpoints, dispatched callbacks, and two effect stores. A ten-run durable-checkpoint experiment shows that a cached admission decision permits checkpoint recreation after deletion. An 80-run cross-process study finds that durable operation-level admission blocks late writes and access to restored checkpoints while preserving the independent root; restored bytes remain present. The contribution is executable adapter evidence and explicit assurance boundaries, building on existing revocation and closure semantics. All traces are controlled and model-free; no original attack helpers are executed. Results establish correspondence within the declared local profiles, rather than production security, physical erasure, original benchmark attack success, or independent audit assurance.

KEYWORDS

AI agents, decommissioning, authorization revocation, security verification, residual state, reproducibility

1 INTRODUCTION

An agent can be marked retired while a separate worker retains a submitted request, a backend accepts an unchecked captured token, or a local registration remains active. A security assessment must therefore distinguish an administrative status transition from observable enforcement and from the state left behind. Our motivating question is concrete: after a retirement action reports success, can an old root still produce an effect, and is there sufficient evidence to close its remaining obligations?

A 2026 government review reports 76 papers aligned with end-of-life risks and an absence of explicit model-disposal keywords in its analysis [1]. Its stated literature scope ends in January 2026. A separate decommissioning review retains seven studies under its own governance-oriented selection [2]. Neither count establishes that technical agent retirement remains unstudied. Later work includes EPITAPH, root-scoped quiescence, VERA, hierarchical credentials, and bounded-closure software [3–6, 8]. A claim to invent agent offboarding, sink fencing, or three-valued verification would consequently be untenable.

We instead examine a narrow engineering question: what changes when an existing closure verifier consumes observations from a concrete local store, and when those observations are incomplete or deliberately stale? We implement an inert fixture with persistent root state, synthetic tokens, queue records, residual records, and an independently launched replay worker. This permits direct measurement without invoking a model or executing public attack helpers.

The contributions are (1) an executable, source-attributed local microbenchmark separating admission and cleanup; (2) raw state and adapter-produced evidence evaluated by an unchanged public verifier; and (3) framework conformance experiments covering dispatched work, checkpoint recreation, cross-process admission, and checkpoint-only recovery. The framework makes each certificate's observation boundary explicit. Its public-case labels do not make the synthetic fixture representative of the original benchmark or of deployed systems. Its durable admission wrapper is an experimental implementation of prior authority-fencing ideas, rather than a new revocation calculus.

2 RELATED WORK AND THE RESEARCH GAP

2.1 Retirement and authority semantics

EPITAPH describes verifiable joint retirement across credentials, tools, memory, policy, and trust, and reports a large retirement simulation [3]. We could inspect its institutional abstract, but its full methodology was inaccessible. We cannot establish which implementation details it lacks. This unresolved closest comparator prevents an exclusive novelty claim.

Root-scoped quiescence formalizes revocation under delegation and asynchronous execution, including sink fences and indeterminate evidence; its reported provider-free cases do not establish concrete provider conformance [4]. VERA studies authority-preserving edge revocation and controlled workflow traces, while identifying production propagation as an open issue [5]. Heartbeat-bound credentials address descendant validity, subject to clock and key assumptions [6]. These results motivate adapter testing; they do not justify treating our simpler local fixture as a reproduction of their protocols.

Authorization succession already studies replacement, forking, and rollback [7]. Our checkpoint-only restoration is a bounded recovery test; replacement, agent forking, and restoration of the authority database are not evaluated here. The public bounded-agent-closure release implements four-domain evidence and CLOSED,

INCOMPLETE, and UNKNOWN outcomes, with canonical fixtures [8]. We reuse that release unchanged. Our adapter is additional experiment code, not a new verification calculus.

2.2 Runtime enforcement and benchmark evidence

Portico, AIRGuard, and AgentSpec already examine runtime controls and enforcement boundaries [9–11]. AgentDojo and ASB provide agent-security tasks rather than retirement ground truth [12, 13]. AgentTrap provides third-party skill trust-failure cases, including persistent or credential-related behavior [14]. A benchmark contract audit cautions that a tool's reported outcome can diverge from actual application state [15]. We therefore score stored effects and artifact state instead of accepting status strings as success evidence.

2.3 A bounded candidate gap

The candidate gap is empirical correspondence between a declared closure profile and concrete observations, especially when cleanup reports success but state remains, or a source advertises freshness it does not provide. Our literature inspection covered eleven full manuscripts, one government report, two partial scholarly records, and one software release. It was a targeted review, not a systematic or exhaustive review. Several author artifacts could not be accessed. The gap should be framed as an opportunity for additional reproducible adapter evidence; absence of accessible artifacts does not prove absence of prior work.

3 QUESTIONS AND HYPOTHESES

RQ1: In the declared local fixture, which controls reject old-root replays while allowing an independently authorized root? **H1:** A durable root fence at the protected sink rejects old-root attempts across the three assigned admission modes, whereas token revocation and a foreground hook leave their deliberately unmediated paths available.

RQ2: Does successful admission blocking imply that residual cleanup completed? **H2:** Paired failed-cleanup variants retain active residual records despite identical old-root admission outcomes under the overlay.

RQ3: How do acknowledgement-only, snapshot, and direct-readback assessments differ under stale and missing evidence? **H3:** A false freshness claim permits false CLOSED outcomes, while faithful readback exposes registered residuals and missing coverage prevents certification of otherwise clean state.

RQ4: Can a delayed checkpoint writer or checkpoint-only restoration regain access after retirement? **H4:** Cached admission permits the selected delayed writes; operation-level admission serialized with durable retirement prevents those mediated writes and denies access to restored checkpoints, while retaining independent-root availability.

These hypotheses concern designed mechanisms and are partly consequences of the implementation. The experiment checks executable correspondence and produces counterexamples to overly broad status-based claims; it does not estimate naturally occurring failure probabilities. Each measured extension records its source hash and protocol before execution, following excluded development runs. None was externally preregistered. RQ4 was added after the first fixture and framework experiments; it is a prospective extension, not an original preregistered hypothesis.

Table 1: Study profiles. Every measured setting has five repetitions. The replication repeats the fixture and supplies no new distinct scenarios.

Profile	Settings	Repeats	Runs
Initial fixture	144	5	720
Separate replication	144	5	720
Framework integration	16	5	80
Durable thread race	2	5	10
Process and recovery	16	5	80

4 FIXTURE AND THREAT MODEL

The fixture contains a retiring root A, an independent root B, root epochs, synthetic token-validity flags, a queued-job row, and one inert residual JSON record. Each request specifies a root, epoch, admission mode, and route. A fresh Python interpreter processes one request from each root after retirement. Accepted requests insert effects into SQLite. The decision and effect insertion share a `BEGIN IMMEDIATE` transaction.

For the overlay, the local admission rule rejects a cut root or an epoch mismatch. This reuses root-fence semantics:

$$\text{allow}(r) = \neg\text{cut}(r.\text{root}) \ \wedge \ (r.\text{epoch} = \text{currentEpoch}(r.\text{root})).$$

The rule applies only at the fixture's protected sink. Root identifiers and route fields are trusted harness inputs, not cryptographically authenticated claims. A real integration would require authenticated attribution and complete sink mediation. The test worker cannot forge another root in the measured scenarios; no security claim against identity forgery is made.

The attacker model is an explicit old-root replay after the retirement transaction. Admission modes are issuer recheck, unchecked cached token, and local uncredentialed access. The latter two deliberately model paths outside token-issuer enforcement. Foreground/background routes similarly delimit the hook's mediation. All modes and routes are assigned by the experimenter; they are not inferred benchmark labels.

The queue is inventory state. Workers replay explicitly even if its row was cancelled, testing the sink against an attempt independent of successful cleanup. We do not measure a scheduler's cancellation delivery. Likewise, a shell-hook or memory label is represented by a JSON record, not an actual shell hook, vector database, or provider memory service. No original skill code, real credential, remote exfiltration, or LLM is executed.

5 DATA AND EXPERIMENTAL METHOD

Table 1 summarizes the separately versioned experiments. Settings are authored control/scenario or factor combinations; none is an independently sampled attack population. Different profiles evaluate different endpoints and are not pooled into one accuracy score.

Table 2: Source-attributed adaptations. Each row is implemented as the same inert residual-record interface; labels do not denote real backend integrations.

Case	Source skill	Local residual label
1	pptx	credential copy
5	video-downloader	shell hook
23	docx	webhook registration
39	mcp-builder	OAuth cache
43	memory-systems	memory metadata
203	finance-news	cron entry
207	moltpixel	heartbeat entry
209	everclaw	upload registration
5005	tmux	terminal hook
5010	self-improving-agent	credential heartbeat
5204	topic-monitor	monitor enrollment
5205	mcp-builder	callback token cache

5.1 Public-data acquisition and selection

AgentTrap supplies 141 task records. The pinned Hugging Face revision is `ab13f59d3f1236e7e65224d4fd6cc00dcdaaf68b`; its task file, manifests, skill files, and fixture archives were downloaded. All 1,474 acquired skill files matched their recorded hashes. The repository revision is `613c37512d16a92731d6f145f67f735f1ced9d20`. Component-specific licensing applies; our replication package does not redistribute original skill code.

We purpose-select twelve cases whose plans contain useful credential, persistence, memory, or registration cues. Table 2 identifies the source and our inert adaptation. This selection is not random and has no matched original-task control. Each becomes a clean and a failed-cleanup variant, giving 24 distinct scenarios. The original skill's attack behavior and success signal are not executed or scored.

Other public artifacts were acquired and audited for future work. AgentDojo's pinned static source contains 86 user-task classes and 27 injection-task classes; these are not the paper's runtime-generated task count. ASB contributes 51 prompts in ten task rows and 400 attack-tool rows, but one apparent JSONL file is actually an upstream rate-limit HTML response and is excluded. AIRGuard's 150-record DTAP index is an index, not a fully runnable benchmark. None of these three artifacts supplies the ground truth used in this experiment.

5.2 Six controls

Administrative sets only a retired flag. *Controller-stop flag* also sets a stopped-controller flag; it models a controller-local status action and does not terminate an operating-system process. The separate replay worker remains available. *Token revocation* invalidates A's synthetic token and blocks its issuer state, affecting issuer-recheck requests. *Foreground hook* blocks A only on the foreground route. *Sink overlay* invalidates the token, advances/cuts A's root state, cancels its queue row, and attempts residual cleanup while enforcing the durable root fence. *Blanket deny* uses the same cleanup but rejects both roots after retirement.

There are four source cases per admission mode and four foreground versus eight background cases. Consequently, each partial control rejects eight of the 24 paired old-root scenarios by construction. Route and mode assignments are correlated; this design cannot separately estimate interactions between them. No released revocation protocol is implemented under a baseline name. These are fixture-level ablations.

5.3 Faults and evidence feeds

Failed cleanup deliberately leaves the residual's active marker true while returning a success acknowledgement. Four of the twelve failed variants also inject a stale snapshot that reports this active artifact as inert. Two source cases, in both variants, declare partial source coverage. These fault frequencies are design choices, not estimates from AgentTrap.

An acknowledgement-only assessment returns CLOSED for every reported-success retirement. Snapshot and readback feeds generate bundles for the unchanged bounded-agent-closure v0.3 verifier. They bind authority, execution, commitment, and operational state to the single local inventory source, which is sufficient only for this fixture's declared profile. Snapshot faults advertise zero bounded visibility lag despite lying about artifact state. Direct readback rereads the registered local state; it cannot repair an explicitly missing source.

Two observations carry logical stamps 110 and 120 following cut stamp 100. These encode deterministic order, not elapsed milliseconds or a measured visibility bound. There is no distributed propagation or wall-clock stability experiment. Evidence references are local labels, not signed attestations.

5.4 Execution, oracle, and metrics

Each run starts from a new database and performs two positive-calibration requests before retirement. Both roots must be admitted. Retirement then precedes the fresh worker's two replay requests. Six controls, 24 scenarios, and five repetitions yield 720 runs, 2,880 total requests including calibration, and 1,440 evidence evaluations. Each control contributes 120 old-root and 120 independent-root post-retirement attempts.

Ground truth independently reads the effect table, queue row, and residual file without calling the adapter. A closure blocker is any old-root post-retirement effect, active residual, or queued original job. False closure means a CLOSED assessment despite such a blocker. These oracle and adapter implementations share the same controlled store and author; they are separate code paths, not independent external audits. An inactive marker represents fixture-level extinguishment, not physical erasure of credential or memory bytes.

We report counts and distinct-scenario counts. Repetitions check deterministic reproducibility and supply descriptive timing samples; they are not independent attacks. We report no population confidence intervals or significance tests. Decision timing covers opening the database, locking, state reads, and admission decision before request/effect insertion and commit. Worker duration also includes interpreter startup and two requests. Fixed control order on one Windows host prevents causal overhead claims.

Table 3: Actual effect counts. Each control has 120 runs: 24 distinct scenarios repeated five times. Independent root B has one post-retirement request per run.

Control	Old A	Independent B	Blockers
Administrative	120	120	120
Controller-stop flag	120	120	120
Token revocation	80	120	120
Foreground hook	80	120	120
Sink overlay	0	120	60
Blanket deny	0	0	60

Table 4: Overlay closure assessments over 120 runs. Divide each entry by five for distinct-scenario counts. False CLOSED is a subset of CLOSED.

Evidence	Closed	Incomplete	Unknown	False
Acknowledgement	120	0	0	60
Snapshot	70	40	10	20
Direct readback	50	60	10	0

6 RESULTS

6.1 Admission and collateral blocking

Table 3 gives actual stored post-retirement effects. The overlay admits zero old-root requests and all independent-root requests, corresponding to zero of 24 and 24 of 24 distinct scenarios respectively. Token revocation and the foreground hook each admit old-root effects in 16 of 24 scenarios (80 of 120 repeated runs). Administrative and controller-stop status changes admit all old-root effects. Blanket denial rejects both roots, demonstrating the value of measuring preservation alongside rejection.

H1 holds within the fixture, as expected from the configured mediation boundaries. It does not establish superiority over deployed token revocation, where a backend might recheck every request, or over a runtime hook that mediates all routes. All five repetitions agree on binary effects and verdicts for every control/scenario pair.

6.2 Cleanup and false certification

All twelve failed-cleanup scenarios retain active synthetic residuals under the overlay, while all twelve clean variants clear the marker. Both groups have identical old-root rejection. Thus H2 holds: admission safety alone does not certify cleanup.

For the overlay, the acknowledgement-only assessment falsely closes all twelve failed scenarios. Snapshot assessment falsely closes the four deliberately stale failed scenarios. Direct readback falsely closes none. Table 4 reports repeated-run counts, including the two clean scenarios with partial coverage. Known active residuals produce INCOMPLETE even when coverage is partial; absent blockers plus partial coverage produce UNKNOWN.

Across all controls, acknowledgement-only assessment falsely closes 600 of 720 runs; snapshot assessment falsely closes 40, split equally between overlay and blanket denial. Readback has zero false CLOSED outcomes in the designed corpus. The stale failures reflect an adapter violating its advertised freshness contract, not a demonstrated defect in the unchanged verifier. Direct readback detects the specific registered artifact because it reads the same authoritative fixture state as the oracle. It does not prove completeness for hidden or unregistered artifacts. H3 is supported within that boundary.

6.3 Validation and descriptive timing

Five local boundary tests passed, including durable-fence readback in a fresh interpreter, independent-root preservation, cached-token bypass of issuer-only revocation, false acknowledgement versus raw state, blanket collateral blocking, and unknown-root rejection (some checks share a test). The upstream verifier previously passed 75 tests and eight canonical fixtures. All 1,440 final evidence bundles pass its schema validation. A development pilot exposed an enum mismatch; repair and pilot schema validation preceded the final measured execution. The pilot and an interrupted earlier measured directory are excluded.

The overlay's median decision time is 2.7199 ms and descriptive p95 is 3.5843 ms, compared with administrative median 2.7222 ms and p95 3.6759 ms. Median overlay worker invocation is 156.4491 ms. The apparent small differences are not a causal overhead estimate: the measurements exclude effect insertion/commit from decision timing and include host/process variability. No latency hypothesis is tested.

6.4 Separate complete replication

A second 720-run execution in a new result directory repeats the original fixture without changing its measured implementation. A comparison keyed by scenario and control verifies all 720 semantic outcomes and all 1,440 verdicts against measured-v2, excluding timing. All agree. The replication uses a separately recorded design hash and is reported separately; it does not double the number of distinct scenarios or add independent attack samples.

7 FRAMEWORK, CONCURRENCY, AND RECOVERY EXTENSIONS

The following experiments are separate, versioned extensions of the initial fixture. They use actual LangGraph checkpoint APIs and APScheduler job APIs [16, 17]. They exercise authored lifecycle wrappers with synthetic identities. A permitted invocation recreating a deleted checkpoint is expected framework behavior; these experiments do not establish a library vulnerability.

7.1 Dispatched callbacks and two effect stores

LangGraph 1.2.14 with InMemorySaver and APScheduler 3.11.3 issue signed requests to a loopback HTTP sink backed by SQLite transactions or append-only JSONL. Sink fencing, checkpoint/job cleanup, and a sequential graph admission gate are independently varied. Two stores, eight factor settings, and five repetitions yield 80 runs. A dispatched callback waits at a barrier while retirement occurs. A second pending job is removed through the scheduler API when cleanup is enabled; otherwise it is rescheduled to execute after the cut. Both roots calibrate before retirement. Per-run ephemeral HMAC keys bind request root and phase; all 80 deliberately invalid signatures are rejected. The harness controls identities and keys, so this does not test a production identity boundary.

Table 5: Framework results. Each row contains ten runs: two stores and five repetitions. Checkpoint is presence after late invocation; effects count committed retired-root requests.

Fence	Cleanup	Graph gate	Effects	Checkpoint
Off	Off	Off	30	10
Off	Off	On	20	10
Off	On	Off	20	10
Off	On	On	10	0
On	Off	Off	0	10
On	Off	On	0	10
On	On	Off	0	10
On	On	On	0	0

Table 5 separates committed effects from the presence of an in-memory checkpoint after a late invocation. Forty fenced runs commit no retired-root effects, while forty unfenced runs commit 80. All 80 independent-root post-cut effects commit. Cleanup deletes the checkpoint at an intermediate readback, but an unguarded late invocation recreates it in all 20 clean, ungated runs. Cleanup plus the sequential gate prevents recreation in all 20 clean, gated runs. Sink admission still needs to cover dispatched work: a graph gate alone does not cancel an already admitted callback.

The effect oracle uses a separate reader of the raw SQLite/JSONL stores and verifies their hashes. InMemorySaver checkpoint observations remain harness-reported because that state disappears with the process. The sequential gate is not atomic with graph execution; the next extensions explicitly address that boundary.

7.2 Durable checkpoint admission race

Ten runs use LangGraph SqliteSaver 3.1.1, two admission controls, and five repetitions. Each creates checkpoints for A and B and retains a consistent SQLite backup. A worker checks A's authority and pauses. Retirement invalidates A and calls `delete_thread`; raw SQLite reads confirm zero live A checkpoint rows. Releasing a worker that relies on its earlier check recreates A checkpoints in five of five runs. Rechecking authority while holding the same thread lifecycle lock through the entire invocation prevents recreation in all five guarded runs. Fresh connections confirm final counts; B's count is unchanged in all ten runs. All ten backups retain the original A checkpoints.

This selected time-of-check/time-of-use interleaving demonstrates why an admission observation alone cannot establish later write safety. Holding a lock throughout graph execution can delay retirement; no latency or liveness claim follows. The lock covers threads within one process, motivating a durable authority boundary for the next experiment.

7.3 Cross-process admission and checkpoint-only recovery

The process extension varies four controls (unmediated access, a cached admission check, operation-level durable admission, and blanket denial), two interleavings (pause before invocation or inside a graph node), and two recovery conditions (live store or restoration of a pre-retirement checkpoint backup). Each worker is a fresh Python process. The protocol excludes development runs. Five repetitions per configuration produce 80 runs, representing 16 configurations rather than 80 independent attacks.

An authority SQLite database is separate from the checkpoint database. A wrapper binds a trusted root and epoch zero to a SqliteSaver instance and mediates synchronous `get_tuple`, `put`, and `put_writes`. Each operation takes an authority `BEGIN IMMEDIATE` transaction, checks active state and epoch, and holds that transaction until its checkpoint operation completes. Retirement takes the same authority transaction, invalidates A, advances its epoch, commits the cut, and deletes A's checkpoints. Thus an operation serialized before the cut can complete before retirement, while one serialized after it is denied. This is a local serialization mechanism, not a new distributed proof. Async methods, listing, bypass callers, and authority-store rollback are outside the wrapper's evaluated profile.

The mid-graph barrier allows an already admitted node to finish after retirement; the guard targets its checkpoint operations, not arbitrary node side effects. For recovery cases, the parent restores only the checkpoint database from its pre-retirement backup before releasing the worker. The separate authority database remains retired. A fresh process then probes A's checkpoint access and invokes B. Raw readers inspect all databases and the authority audit sequence. Retained backup bytes and logical access are recorded separately.

Two databases do not provide one atomic crash transaction: a process failure between checkpoint commit and authority audit commit can leave incomplete evidence. This experiment forces normal-completion interleavings and does not test power loss, crashes, durable storage corruption, authority rollback, or distributed partitions. Every checkpoint caller must use the wrapper for its admission property to apply. A caller with direct database access can bypass it.

7.4 Process and recovery results

All 80 measured runs complete with the predefined outcomes. Table 6 aggregates each control across the four interleaving/recovery combinations and five repetitions. Unmediated and cached admission each allow all 20 delayed invocations and all 20 subsequent checkpoint-access probes. Durable operation-level admission denies all 20 delayed invocations and all 20 access probes while allowing all 20 independent-root invocations. Blanket denial also rejects the retired root, but blocks all 20 independent-root invocations.

The 20 durable-admission runs include ten live-store and ten checkpoint-restoration runs. In the live-store cases, A remains absent after deletion; in all ten restoration cases its original rows remain present but inaccessible through the wrapper. The same pattern holds for blanket denial. All 80 retained backups still contain A checkpoints. A fresh raw-data verifier checks three database hashes per run, final checkpoint counts, root epochs, and authority audit sequences. No accepted mediated A operation occurs after the recorded cut. Unmediated accesses have no corresponding wrapper audit and are evaluated by checkpoint readback. H4 holds for the declared synchronous methods and two forced interleavings, conditional on trusted attribution and preservation of the separate authority store.

Table 6: Cross-process results, twenty runs per control. Late counts completed retired-root invocations; Access counts readable retired checkpoints; *B* counts completed independent invocations. Restored rows counts ten recovery runs per control with physically present *A* checkpoint rows at final readback.

Control	Late	Access	*B*	Restored rows
Unmediated	20	20	20	10
Cached admission	20	20	20	10
Durable admission	0	0	20	10
Blanket denial	0	0	0	10

8 DISCUSSION AND VALIDITY LIMITS

The useful engineering finding is the separation of three observables: an attempted effect can be rejected, cleanup can still fail, and evidence can still wrongly report closure. A UI that collapses them into one green success indicator would hide the designed failure cases. Within this fixture, the appropriate certificate boundary is the declared source profile plus its observation contracts, not a retirement API acknowledgement.

Construct validity is limited. In the initial fixture, public cases supply mechanism cues while all residual labels use one JSON interface. Token validity is a Boolean, not OAuth revocation, and the controller-stop control does not kill a process. The extensions exercise a real scheduler and durable checkpoint restoration, but do not evaluate vector-memory deletion, credential-byte sanitization, descendant identity chains, multi-parent support, policy retirement, or inter-agent trust removal. Different initial case labels do not create different backend coverage. The JSONL and SQLite effect stores are separately implemented local adapters authored by the same researcher, not independently audited vendor backends.

Internal validity is also bounded. Initial fixture retirement and replay are sequential. The extensions force specific thread/process interleavings using barriers, rather than exploring every schedule. The fixture trusts root attribution and admits requests through one protected sink. Snapshot faults and readback behavior are deliberately authored. The store is shared by scoring and observation, making the result a correspondence check rather than an independent security audit. Fixed arm order and correlated initial mode/route assignments limit performance and interaction interpretation. Expected outcomes helped design the implementation; passing assertions demonstrate conformance to that design, not discovery of naturally occurring prevalence.

External validity is narrow. The twelve source cases are purpose-selected, and the 24 scenarios are synthetic adaptations rather than original AgentTrap executions. Five repetitions do not enlarge the attack population. The scheduler is an application-level APScheduler instance. No LLM-backed runtime, vendor API, network partition, asynchronous visibility lag, operating-system scheduler, or production IAM service was tested. Outcomes cannot be translated into original benchmark scores or deployment-wide failure rates. Logical denial of restored checkpoints does not establish physical erasure, cryptographic destruction, removal of model influence, or complete agent closure.

Kubernetes' execution-scoped token example already combines run-exit revocation and preserving a subsequent run; its documentation identifies surviving-process and in-memory-revocation restart risks [18]. These generic lifecycle observations are therefore not claimed as new here.

Finally, novelty remains conditional. Our targeted review is not exhaustive, and EPITAPH's full text plus several comparator artifacts were inaccessible. Sink fences, authority preservation, three-valued outcomes, and epoch handling are prior art. The defensible increment is an inspectable sequence of local conformance experiments connecting these concepts to concrete API calls, raw observations, and recovery boundaries. Stronger claims need closest-work methods, independent traces and review, and heterogeneous production integrations. A completed manuscript does not establish acceptance or field-wide novelty.

9 REPRODUCIBILITY AND RESEARCH PRACTICE

A retrospective validation on 20 October 2026 verified all 5,405 original archive member hashes, passed five Python and 75 original verifier tests, and reexecuted the 720-run fixture in an isolated copy. All security outcomes and 1,440 closure verdicts matched; timing was excluded. Separate SQL recounts agreed with both original and fresh fixture records, and the final 80 process-recovery cases passed raw database readback. The framework extensions were not freshly reexecuted in this validation, and no independent external review is implied.

The canonical project retains pinned acquisition manifests, source hashes, derived case records, code, raw per-run SQLite files, residual JSON, evidence bundles, verifier receipts, and machine-readable analysis. The final design hash is

```
276eb9f4ce180f0142343af2e44e473660e3
579f475433d98669192ca6097788.
```

The design file records source attribution, code hashes, environment, controls, fault assignments, and repetitions before final execution. The package provides commands to rerun tests and generate a new result directory. A public technical-report package is provided at https://github.com/ezekielologunde/ai-security-artifact-audits/tree/main/reports/P01. The canonical source repository remains private; the package supplies an explicit reproduction inventory independently. Copied AgentTrap task descriptions are excluded: a restore script downloads the pinned ledger from its original host, reconstructs 722 reference files and checks their original SHA-256 hashes. Raw authored SQLite evidence and the verifier's Apache-2.0 notices are retained. No archival DOI or peer-reviewed acceptance is claimed.

Original downloaded attack instructions are treated as data. The experiment runs only locally authored inert requests and synthetic canaries, with no private credentials or outbound attack payloads. Upstream licensing and source attribution are preserved. The manuscript and implementation were prepared with AI assistance; the human author must review source fidelity, venue disclosure requirements, and all claims before submission. This publication copy names Ezekiel Ologunde and omits contact and affiliation metadata. No venue, acceptance, institutional ethics approval, or DOI is implied by the ACM layout.

10 CONCLUSION

The experiments demonstrate why admission safety, residual cleanup, checkpoint access, and closure evidence require separate observations. A retirement acknowledgement can coexist with accepted effects; a protected sink can reject effects while residual state remains; and a deleted checkpoint can be recreated by delayed work. Preserving retirement authority outside a checkpoint-only restore provides a testable logical admission boundary, while the restored bytes remain observable. The contribution is a reproducible conformance framework with explicit scope, calibrated independent-root behavior, direct artifact reads, and versioned evidence. Authenticated attribution, independent evaluation, crash recovery, and heterogeneous production backends remain necessary before making deployment-wide assurance claims.

REFERENCES

[1] UK Department for Science, Innovation and Technology. 2026. *Thematic review and gap analysis on AI security*. July 10. https://www.gov.uk/government/publications/thematic-review-and-gap-analysis-on-ai-security/thematic-review-and-gap-analysis-on-ai-security.

[2] Laily Indaryani. 2026. *AI System Decommissioning: A Systematic Literature Review*. DOI:10.37595/mediainfo.v25i1.505. Author-uploaded indexed text consulted; publisher full text unresolved.

[3] Mohamed Chahine Ghanem. *EPITAPH: Verifiable Decommissioning of Stateful Agentic Systems*. 2026. SSRN 7341378. Abstract consulted; full methods remain inaccessible. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7341378.

[4] Genliang Zhu and Chu Wang. *Authorization Revocation for Long-Running AI Agents: Root-Scoped Quiescence under Delegation and Asynchronous Execution*. 2026. arXiv:2609.21284v1. https://arxiv.org/abs/2609.21284.

[5] Lifei Liu, Haoran Yu, and Xiaochong Jiang. *VERA: Authority-Preserving Edge Revocation for Federated AI-Agent Workflows*. 2026. arXiv:2608.30091v1. https://arxiv.org/abs/2608.30091.

[6] Saurabh Deochake. *Heartbeat-Bound Hierarchical Credentials: Cryptographic Revocation for AI Agent Swarms*. 2026. arXiv:2605.20704v1. https://arxiv.org/abs/2605.20704.

[7] Genliang Zhu and Chu Wang. *Authorization for Self-Modifying AI Agent Populations: Conserving Authority across Replacement, Forking, and Rollback*. 2026. arXiv:2610.00347v1. https://arxiv.org/abs/2610.00347.

[8] RISU Research. 2026. *bounded-agent-closure*, v0.3.1. Commit ef072360b7778e6e38a299576cbc2b8d7ac62f02. Apache-2.0. https://github.com/risu-research/bounded-agent-closure.

[9] Igor Santos-Grueiro. *Lingering Authority: Revocable Resource-and-Effect Capabilities for Coding Agents*. 2026. arXiv:2606.22504v1. https://arxiv.org/abs/2606.22504.

[10] Suliu Qin et al. *AIRGuard: Guarding Agent Actions with Runtime Authority Control*. 2026. arXiv:2605.28914v1. https://arxiv.org/abs/2605.28914.

[11] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. *AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents*. 2025. arXiv:2503.18666v1. https://arxiv.org/abs/2503.18666.

[12] Edoardo Debenedetti et al. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. 2024. arXiv:2406.13352v1. https://arxiv.org/abs/2406.13352.

[13] Hanrong Zhang et al. *Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents*. 2025. ICLR; manuscript arXiv:2410.02644v3. https://arxiv.org/abs/2410.02644.

[14] Haomin Zhuang et al. *AgentTrap: Measuring Runtime Trust Failures in Third-Party Agent Skills*. 2026. arXiv:2605.13940v1. Data: https://huggingface.co/datasets/zhmzm/AgentTrap. Manuscript: https://arxiv.org/abs/2605.13940.

[15] Rohith Reddy Bellibatlu, Zichong Wang, and Wenbin Zhang. *Do Agent Benchmarks Do What They Say? An Executable-Contract Audit of Tool-Using Agent Environments*. 2026. arXiv:2609.37315v1. https://arxiv.org/abs/2609.37315.

[16] LangChain. 2026. *LangGraph SqliteSaver API reference*. Accessed October 7. https://reference.langchain.com/python/langgraph.checkpoint.sqlite/SqliteSaver.

[17] APScheduler contributors. *APScheduler 3.x user guide*. Accessed October 7, 2026. https://apscheduler.readthedocs.io/en/stable/userguide.html.

[18] Kubernetes SIGs. 2026. Execution-Scoped Token Example. Inspected commit `b03821779de0b32572bbd13a4661a983bfd8cfdd`. https://agent-sandbox.sigs.k8s.io/docs/use-cases/examples/containarium-execution-scoped-token/.